```
#Minmax Algorithm----------------------------------------------------
def minimax(depth, node, maximizing, values):

    # If we reach leaf node
    if depth == 2:
        return values[node]

    # MAX player
    if maximizing:
        left = minimax(depth + 1, node * 2, False, values)
        right = minimax(depth + 1, node * 2 + 1, False, values)

        return max(left, right)

    # MIN player
    else:
        left = minimax(depth + 1, node * 2, True, values)
        right = minimax(depth + 1, node * 2 + 1, True, values)

        return min(left, right)


values = [3, 5, 2, 9]

result = minimax(0, 0, True, values)
```

```
print("Best value:", result)
```

#Apha beta pruning----------

```
def alpha_beta(depth, node, maximizing, values, alpha, beta):

    if depth == 2:
        return values[node]

    if maximizing:

        best = float('-inf')

        for i in [0, 1]:

            value = alpha_beta(
                depth + 1,
                node * 2 + i,
                False,
                values,
                alpha,
                beta
            )

            best = max(best, value)
            alpha = max(alpha, best)
```

```
        if beta <= alpha:
            break

    return best

else:

    best = float('inf')

    for i in [0, 1]:

        value = alpha_beta(
            depth + 1,
            node * 2 + i,
            True,
            values,
            alpha,
            beta
        )

        best = min(best, value)
        beta = min(beta, best)

        if beta <= alpha:
            break
```

```
    return best


values = [3, 5, 2, 9]


result = alpha_beta(
  0,
  0,
  True,
  values,
  float('-inf'),
  float('inf')
)


print("Best value:", result)
```

Genetic Algorithm

```
import random

# Convert binary chromosome to decimal
def binary_to_decimal(chromosome):
    return int(chromosome, 2)


# Fitness function
def fitness(chromosome):
    x = binary_to_decimal(chromosome)
    return x * x
```

```
# Create initial population
population = [
    "10101",
    "11000",
    "01101",
    "11100"
]


# Selection
def selection(population):

    population.sort(key=fitness, reverse=True)
    return population[:2]


# Crossover
def crossover(parent1, parent2):
    point = 2

    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]

    return child1, child2


# Mutation
def mutation(chromosome):
    chromosome = list(chromosome)

    position = random.randint(0, len(chromosome) - 1)

    if chromosome[position] == '0':
        chromosome[position] = '1'
    else:
        chromosome[position] = '0'

    return ''.join(chromosome)
```

```
# Main Genetic Algorithm
for generation in range(5):

    print("\nGeneration:", generation + 1)

    # Display population and fitness
    for individual in population:
        print(
            individual,
            "x =", binary_to_decimal(individual),
            "fitness =", fitness(individual)
        )

    # Select best two
    parents = selection(population)
    print("Parents:", parents)

    # Crossover
    child1, child2 = crossover(parents[0], parents[1])

    # Mutation
    child1 = mutation(child1)
    child2 = mutation(child2)

    # Create new population
    population = parents + [child1, child2]


# Find best solution
best = max(population, key=fitness)

print("\nBest Solution:", best)
print("Best x:", binary_to_decimal(best))
print("Best Fitness:", fitness(best))
```

BFS – Python Code

```
from collections import deque


# Graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}


def bfs(graph, start):

    visited = set()
    queue = deque([start])

    while queue:

        node = queue.popleft()

        if node not in visited:

            print(node, end=" ")
            visited.add(node)

            for neighbour in graph[node]:

                if neighbour not in visited:
                    queue.append(neighbour)

# Call BFS
print("BFS Traversal:")
bfs(graph, 'A')
```

DFS – Python Code

```
# Graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}


def dfs(graph, node, visited):

    if node not in visited:

        print(node, end=" ")
        visited.add(node)

        for neighbour in graph[node]:

            dfs(graph, neighbour, visited)


# Visited set
visited = set()

print("DFS Traversal:")
dfs(graph, 'A', visited)
```

A* Search – Python Code

```
import heapq


def a_star(graph, heuristic, start, goal):
```

```
    queue = [(0, start)]
    cost = {start: 0}

    while queue:

        f, current = heapq.heappop(queue)

        print("Visiting:", current)

        if current == goal:

            print("Goal found!")
            return

        for neighbour, distance in graph[current]:

            g = cost[current] + distance
            f = g + heuristic[neighbour]

            if neighbour not in cost or g < cost[neighbour]:

                cost[neighbour] = g

                heapq.heappush(queue, (f, neighbour))


graph = {
    'A': [('B', 2), ('C', 1)],
    'B': [('G', 2)],
    'C': [('D', 5)],
    'D': [('G', 1)],
    'G': []
}


heuristic = {
    'A': 4,
    'B': 2,
```

```
    'C': 6,
    'D': 1,
    'G': 0
}


a_star(graph, heuristic, 'A', 'G')

#GREEDY-------------------------------------------------------------

import heapq

def greedy(graph, heuristic, start, goal):

    queue = [(heuristic[start], start)]
    visited = set()

    while queue:

        h, current = heapq.heappop(queue)

        if current in visited:
            continue

        print("Visiting:", current)
        visited.add(current)

        if current == goal:
            print("Goal found!")
            return

        for neighbour in graph[current]:
            if neighbour not in visited:
                heapq.heappush(queue, (heuristic[neighbour],
neighbour))


graph = {
    'A': ['B', 'C'],
```

```
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

heuristic = {
    'A': 6,
    'B': 4,
    'C': 2,
    'D': 3,
    'E': 1,
    'F': 0
}

greedy(graph, heuristic, 'A', 'F')
```